

Reparto de decrementos bajo UDD

*Equivalencia entre la formulación integral y el desarrollo de inclusión-exclusión, y
comparación de su coste computacional*

Proyecto:

lifeflow

Fecha:

20 de julio de 2026

Documentación técnica del motor de decrementos

Declaración de intenciones

El motor de `lifeflow` reparte las salidas de cada período entre las causas concurrentes mediante un desarrollo de inclusión-exclusión con 2^{K-1} términos por causa. Este documento demuestra que ese desarrollo no es una definición, sino el resultado de resolver analíticamente una integral, y que dicha integral puede evaluarse de forma *exacta* —no aproximada— con un número de operaciones que crece de forma polinómica en lugar de exponencial.

El objetivo es doble. Primero, justificar matemáticamente la equivalencia con el detalle suficiente para que la decisión de intercambiar los papeles entre ambos métodos —convertir la integral en el núcleo de cálculo y el desarrollo actual en el test de referencia— se tome con conocimiento y no por confianza. Segundo, cuantificar la ventaja computacional y delimitar sus condiciones de validez, porque la exactitud de la cuadratura depende de una relación concreta entre el número de nodos y el número de causas que, de incumplirse, degrada el método sin emitir ningún aviso.

Conceptos Técnicos

Modelo de decrementos múltiples · Distribución uniforme de decrementos (UDD) · Fuerza de decremento · Tasas independientes y dependientes · Principio de inclusión-exclusión · Cuadratura de Gauss–Legendre · Polinomios de Legendre · Complejidad computacional · Compilación JIT

Índice

Índice	2
1. El modelo de decrementos múltiples	3
1.1. Notación	3
1.2. La hipótesis de distribución uniforme	3
1.3. Fuerza de decremento	3
1.4. Supervivencia conjunta	4
2. La probabilidad dependiente como integral	5
2.1. Planteamiento	5
2.2. Simplificación del integrando	5
3. De la integral al desarrollo de inclusión-exclusión	6
3.1. Desarrollo del producto	6
3.2. Integración término a término	6
3.3. Casos particulares	6
3.4. Correspondencia con la implementación	7
4. Evaluación por cuadratura de Gauss–Legendre	8
4.1. Naturaleza del integrando	8
4.2. Exactitud de la cuadratura gaussiana	8
4.3. Número de nodos requerido	8
4.4. Traslado del intervalo	9
4.5. Advertencia sobre la condición de exactitud	9
5. Comparación computacional	10
5.1. Coste del desarrollo de inclusión-exclusión	10
5.2. Coste de la cuadratura	10
5.3. Ratio teórico y contraste empírico	10
5.4. Rendimiento sobre el motor completo	11
5.5. Estabilidad numérica	11
5.6. Sobre la necesidad de compilación JIT	11
6. El caso de fuerza constante	12
7. Conclusiones	13

1. El modelo de decrementos múltiples

1.1. Notación

Consideremos un período de proyección, normalizado al intervalo $[0, 1]$, sobre el que actúan K causas de salida concurrentes: mortalidad, caída, invalidez, o cualquier otra que el actuario decida modelizar.

Cada causa $j \in \{1, \dots, K\}$ viene descrita por su **tasa independiente** q'_j , también llamada tasa de la tabla de decremento simple. Es la probabilidad de que la causa j provoque la salida durante el período *si actuara en solitario*, sin competencia de las demás.

Lo que necesita el motor no es esa tasa, sino la **probabilidad dependiente** $(aq)_j$: la probabilidad de que la causa j sea efectivamente la que provoque la salida, teniendo en cuenta que las otras $K - 1$ causas están compitiendo por el mismo colectivo. Siempre se cumple $(aq)_j < q'_j$, porque parte de quienes habrían salido por la causa j salen antes por otra.

1.2. La hipótesis de distribución uniforme

La hipótesis UDD se formula *sobre cada tabla de decremento simple por separado*: dentro del período, las salidas por la causa j se distribuyen uniformemente en el tiempo.

En términos de supervivencia, si ${}_s p'_j$ denota la probabilidad de no haber salido por la causa j al instante $s \in [0, 1]$:

$${}_s p'_j = 1 - s q'_j \quad (1)$$

Es decir, la supervivencia decrece linealmente. En $s = 0$ vale 1; en $s = 1$ vale $1 - q'_j$, como debe ser.

Conviene subrayar el matiz porque es la fuente habitual de confusión: uniforme significa que sale el *mismo número absoluto* de individuos en cada subintervalo, medido sobre la cohorte inicial. No significa que la tasa condicional sea constante. De hecho, la tasa condicional crece a lo largo del período, precisamente porque la base sobre la que se aplica va menguando.

1.3. Fuerza de decremento

La fuerza de decremento de la causa j en el instante s se define como la tasa instantánea de salida relativa a los supervivientes en ese momento:

$$\mu_j(s) = -\frac{d}{ds} \ln {}_s p'_j = \frac{-\frac{d}{ds} {}_s p'_j}{{}_s p'_j} \quad (2)$$

Aplicando (1), la derivada del numerador es $-q'_j$, con lo que:

$$\mu_j(s) = \frac{q'_j}{1 - s q'_j} \quad (3)$$

El numerador constante refleja el flujo uniforme de salidas; el denominador decreciente refleja que ese flujo se reparte entre cada vez menos supervivientes. La fuerza, por tanto, es creciente en s .

1.4. Supervivencia conjunta

Suponiendo independencia entre las causas —hipótesis estándar del modelo de decrementos múltiples—, la probabilidad de no haber salido por *nninguna* causa al instante s es el producto de las supervivencias individuales:

$${}_s p^{(\tau)} = \prod_{m=1}^K {}_s p'_m = \prod_{m=1}^K (1 - s q'_m) \quad (4)$$

Evaluada en $s = 1$, esta expresión da la supervivencia al final del período, $\prod_m (1 - q'_m)$, que es exactamente lo que calcula la función `_compute_inf` del motor mediante su acumulación multiplicativa.

2. La probabilidad dependiente como integral

2.1. Planteamiento

La probabilidad de salir por la causa j durante el período se obtiene acumulando, sobre todos los instantes s del intervalo, la probabilidad de dos sucesos simultáneos: seguir presente en s pese a todas las causas, y que la causa j actúe justo en ese instante.

$$(aq)_j = \int_0^1 {}_s p^{(\tau)} \cdot \mu_j(s) ds \quad (5)$$

Esta es la **definición**. Todo lo que sigue son consecuencias de ella.

2.2. Simplificación del integrando

Sustituyendo (4) y (3) en (5):

$$(aq)_j = \int_0^1 \left[\prod_{m=1}^K (1 - s q'_m) \right] \cdot \frac{q'_j}{1 - s q'_j} ds \quad (6)$$

El factor $m = j$ del productorio se cancela con el denominador de la fuerza. Este es el punto clave de toda la construcción, y explica por qué la expresión final resulta tan manejable:

$$(aq)_j = q'_j \int_0^1 \prod_{\substack{m=1 \\ m \neq j}}^K (1 - s q'_m) ds \quad (7)$$

Obsérvese la estructura resultante. Fuera de la integral queda la tasa independiente de la causa de interés. Dentro queda únicamente el efecto de las causas *competidoras*: el factor por el cual la presencia de las demás reduce las salidas atribuibles a j .

La cancelación tiene además una lectura probabilística. La causa j retira individuos a ritmo constante q'_j respecto de la cohorte inicial; de ellos, solo cuentan los que en ese instante no han sido ya retirados por otra causa, y esa fracción es precisamente $\prod_{m \neq j} (1 - s q'_m)$.

3. De la integral al desarrollo de inclusión-exclusión

Demostramos ahora que resolver (7) analíticamente produce, término a término, el desarrollo implementado en el motor.

3.1. Desarrollo del producto

Sea $A = \{1, \dots, K\} \setminus \{j\}$ el conjunto de causas competidoras, con $|A| = K - 1$. Desarrollar un producto de binomios equivale a sumar sobre todos los subconjuntos de factores:

$$\prod_{m \in A} (1 - s q'_m) = \sum_{S \subseteq A} \prod_{m \in S} (-s q'_m) = \sum_{S \subseteq A} (-1)^{|S|} s^{|S|} \prod_{m \in S} q'_m \quad (8)$$

De cada binomio se elige el 1 o el término $-s q'_m$; cada elección posible corresponde a un subconjunto S de causas. Los 2^{K-1} subconjuntos de A generan los 2^{K-1} sumandos. El subconjunto vacío aporta el término constante 1.

3.2. Integración término a término

La suma (8) es finita, de modo que puede integrarse término a término. Cada sumando contiene una única potencia de s :

$$\int_0^1 s^{|S|} ds = \frac{1}{|S| + 1} \quad (9)$$

Sustituyendo en (7):

$$(aq)_j = q'_j \sum_{S \subseteq A} \frac{(-1)^{|S|}}{|S| + 1} \prod_{m \in S} q'_m \quad (10)$$

Esta es la fórmula de inclusión-exclusión. Los denominadores $1/2, 1/3, 1/4, \dots$ que aparecen en la implementación no son coeficientes arbitrarios ni correcciones empíricas: son el resultado de integrar $s^0, s^1, s^2, \dots$ sobre el intervalo unidad. Los signos alternados proceden del desarrollo del producto de binomios.

Proposición 1. *Las expresiones (7) y (10) son idénticas. No se ha introducido ninguna aproximación en el paso de una a otra: únicamente el desarrollo de un producto finito y la integración exacta de monomios.*

3.3. Casos particulares

Dos causas

Con $K = 2$ y $A = \{a\}$, los subconjuntos son $\emptyset$ y $\{a\}$:

$$(aq)_j = q'_j \left(1 - \frac{q'_a}{2} \right) \quad (11)$$

Es la fórmula clásica de los manuales. Su interpretación es directa: la causa competidora retira, en promedio a lo largo del período, la mitad de la exposición.

Tres causas

Con $K = 3$ y $A = \{a, b\}$, los subconjuntos son $\emptyset$, $\{a\}$, $\{b\}$ y $\{a, b\}$:

$$(aq)_j = q'_j \left(1 - \frac{q'_a}{2} - \frac{q'_b}{2} + \frac{q'_a q'_b}{3} \right) \quad (12)$$

Aparece el término de segundo orden, que corrige la doble sustracción de los individuos que habrían salido por ambas causas competidoras.

Seis causas

Con $K = 6$ el desarrollo contiene $2^5 = 32$ términos por causa, distribuidos en $1 + 5 + 10 + 10 + 5 + 1$ sumandos de órdenes 0 a 5. Su escritura explícita es inviable en una hoja de cálculo, lo cual motivó buscar una vía de validación alternativa a la comparación contra Excel.

3.4. Correspondencia con la implementación

La función `_udd_correction` del módulo `_inforce_nb.py` recorre los subconjuntos mediante una máscara de bits:

```
correction = 1.0
for mask in range(1, 1 << n_others):
    size = 0
    product = 1.0
    for b in range(n_others):
        if mask & (1 << b):
            size += 1
            product *= others[b]
    sign = 1 if (size % 2 == 0) else -1
    correction += sign / (size + 1) * product
```

La correspondencia con (10) es término a término:

Fórmula	Código
$S \subseteq A$	<code>mask</code> , recorriendo 1 a $2^{K-1} - 1$
$ S $	<code>size</code>
$\prod_{m \in S} q'_m$	<code>product</code>
$(-1)^{ S }$	<code>sign</code>
$1/(S + 1)$	<code>1/(size+1)</code>
Término de $S = \emptyset$	inicialización <code>correction = 1.0</code>
q'_j	factor externo <code>q_vec[j]</code>

El bucle arranca en `mask = 1` porque el subconjunto vacío ya está contabilizado en la inicialización.

4. Evaluación por cuadratura de Gauss–Legendre

Establecida la equivalencia, cabe evaluar (7) numéricamente en lugar de desarrollarla. La cuestión es si ello introduce error.

4.1. Naturaleza del integrando

El integrando de (7) es un producto de $K - 1$ factores lineales en s . Por tanto:

Lema 1. *El integrando $f(s) = \prod_{m \neq j} (1 - s q'_m)$ es un polinomio en s de grado exactamente $K - 1$.*

Esta observación es el fundamento de todo lo que sigue. No se trata de una función arbitraria que haya que aproximar, sino de un polinomio de grado bajo y conocido de antemano.

4.2. Exactitud de la cuadratura gaussiana

Una regla de cuadratura aproxima una integral por una suma ponderada de evaluaciones:

$$\int_{-1}^1 f(x) dx \approx \sum_{k=1}^n w_k f(x_k) \quad (13)$$

Las reglas elementales —rectángulos, trapecios, Simpson— fijan los nodos x_k de manera equiespaciada y solo ajustan los pesos. La cuadratura de Gauss–Legendre optimiza *ambos* conjuntos simultáneamente, tomando como nodos las raíces del polinomio de Legendre P_n .

Proposición 2. *La regla de Gauss–Legendre de n nodos integra exactamente todo polinomio de grado menor o igual que $2n - 1$.*

Esbozo de demostración. Sea p un polinomio con $\deg p \leq 2n - 1$. Por división euclídea entre p y P_n existen q y r con $\deg q, \deg r \leq n - 1$ tales que $p = q P_n + r$. Los polinomios de Legendre son ortogonales a todo polinomio de grado inferior a n , luego $\int_{-1}^1 q P_n = 0$ y por tanto $\int_{-1}^1 p = \int_{-1}^1 r$. Por otro lado, los nodos x_k son raíces de P_n , de modo que $p(x_k) = r(x_k)$ y la suma ponderada de p coincide con la de r . Finalmente, la regla integra exactamente todo polinomio de grado $\leq n - 1$ por construcción de los pesos. Ambos miembros coinciden. $\square$

La palabra *exactamente* debe entenderse en sentido literal: salvo el redondeo inherente a la aritmética de coma flotante, no hay error de truncamiento.

4.3. Número de nodos requerido

Combinando el lema y la proposición, la condición de exactitud es:

$$2n - 1 \geq K - 1 \quad \Longleftrightarrow \quad n \geq \frac{K}{2} \quad (14)$$

de donde

$$\boxed{n = \left\lceil \frac{K}{2} \right\rceil} \quad (15)$$

Para $K = 6$ bastan tres nodos; para $K = 12$, seis.

K	Grado del integrando	Nodos mínimos	Exacto hasta grado	Términos incl.-excl.
2	1	1	1	2
3	2	2	3	4
4	3	2	3	8
6	5	3	5	32
8	7	4	7	128
10	9	5	9	512
12	11	6	11	2048

4.4. Traslado del intervalo

Los nodos y pesos tabulados corresponden al intervalo $[-1, 1]$, mientras que (7) se define sobre $[0, 1]$. El cambio de variable $s = \frac{1}{2}(x + 1)$ da:

$$\int_0^1 f(s) ds = \frac{1}{2} \int_{-1}^1 f\left(\frac{x+1}{2}\right) dx \approx \sum_{k=1}^n \tilde{w}_k f(\tilde{s}_k), \quad \tilde{s}_k = \frac{x_k + 1}{2}, \quad \tilde{w}_k = \frac{w_k}{2} \quad (16)$$

4.5. Advertencia sobre la condición de exactitud

La relación (15) es una condición *necesaria*. Si el número de nodos es insuficiente para el número de causas, la cuadratura deja de ser exacta y pasa a aproximar, sin que ninguna comprobación en tiempo de ejecución lo señale. El error resultante sería pequeño y de aspecto plausible, es decir, difícil de detectar por inspección.

En una implementación de producción, el número de nodos debe derivarse de K mediante (15) y no fijarse como constante.

5. Comparación computacional

5.1. Coste del desarrollo de inclusión-exclusión

Recorrer los 2^{K-1} subconjuntos, evaluando en cada uno un producto de hasta $K - 1$ factores, cuesta por causa:

$$C_{\text{IE}} = \mathcal{O}(2^{K-1}(K - 1)) \quad (17)$$

y para las K causas de una celda, $\mathcal{O}(K^2 2^{K-1})$. El crecimiento es **exponencial**.

A ello se añade un coste no asintótico pero relevante: la implementación reserva un vector auxiliar en cada invocación, lo que en un grid de N pólizas y T períodos supone $N \times T \times K$ reservas de memoria.

5.2. Coste de la cuadratura

Evaluar el integrando en $n = \lceil K/2 \rceil$ nodos, cada uno con un producto de $K - 1$ factores, cuesta por causa:

$$C_{\text{GL}} = \mathcal{O}\left(\frac{K}{2}(K - 1)\right) = \mathcal{O}(K^2) \quad (18)$$

y para las K causas, $\mathcal{O}(K^3)$. El crecimiento es **polinómico**. No requiere memoria auxiliar: los nodos y pesos se calculan una sola vez fuera del núcleo.

5.3. Ratio teórico y contraste empírico

Dividiendo ambas expresiones:

$$\frac{C_{\text{IE}}}{C_{\text{GL}}} \sim \frac{K^2 2^{K-1}}{K^3/2} = \frac{2^K}{K} \quad (19)$$

La medición sobre funciones compiladas con `numba`, promediando 2×10^5 evaluaciones, confirma la predicción:

K	Incl.-excl.	Gauss-Legendre	Ratio medido	$2^K/K$
2	148,6 ns	12,5 ns	11.9×	2.0
4	389,7 ns	62,4 ns	6.2×	4.0
6	1720,6 ns	155,4 ns	11.1×	10.7
8	9401,1 ns	298,5 ns	31.5×	32.0
10	66262,6 ns	621,4 ns	106.6×	102.4
12	963731,3 ns	712,1 ns	1353.3×	341.3

El acuerdo entre las dos últimas columnas es notable para $K \geq 6$. Las desviaciones en los extremos tienen explicación conocida: en $K = 2$ el ratio medido supera al teórico porque la reserva de memoria domina sobre un cálculo trivial; en $K = 12$ lo supera porque los 2048 subconjuntos desbordan la caché del procesador, un efecto que el análisis asintótico no recoge.

5.4. Rendimiento sobre el motor completo

Las cifras anteriores aíslan el núcleo aritmético. Medido sobre `_compute_exits` con una cartera de $N = 20\,000$ pólizas y $T = 240$ períodos:

K	Incl.-excl.	Gauss–Legendre	Mejora
2	0,155 s	0,076 s	$2.0\times$
3	0,275 s	0,130 s	$2.1\times$
6	3,078 s	0,595 s	$5.2\times$

La mejora es menor que en el micro-benchmark porque el resto del núcleo —recorrido del grid, accesos a memoria, escritura de resultados— es común a ambos métodos y no se ve afectado. Los resultados coinciden entre ambos métodos con tolerancia relativa 10^{-12} .

5.5. Estabilidad numérica

El desarrollo (10) es una suma de términos con signos alternados, estructura susceptible en principio de cancelación catastrófica. La cuadratura, en cambio, suma productos de factores en $(0, 1]$ con pesos positivos, sin cancelación posible.

Contrastando ambos métodos contra aritmética racional exacta con tasas deliberadamente extremas (q'_m entre 0,6 y 0,95, muy por encima de cualquier valor actuarial realista) y $K = 8$, los errores relativos resultan ser $8,8 \times 10^{-16}$ para el desarrollo y $3,5 \times 10^{-16}$ para la cuadratura. Con tasas de magnitud actuarial habitual, ambos métodos reproducen el valor exacto sin error apreciable.

La ventaja teórica existe, por tanto, pero es de magnitud despreciable. No constituye un argumento para el cambio.

5.6. Sobre la necesidad de compilación JIT

Cabe preguntarse si el carácter vectorizable de la cuadratura —a diferencia del recorrido de máscaras de bits, difícilmente expresable en operaciones de array— permitiría prescindir de `numba`.

La medición lo desmiente. Una implementación en `numpy` puro de la cuadratura resulta entre 3 y 5,5 veces *más lenta* que la implementación compilada del desarrollo de inclusión-exclusión:

K	<code>numba</code> + incl.-excl.	<code>numpy</code> + Gauss	Penalización
2	0,174 s	0,546 s	$3.1\times$
3	0,292 s	1,619 s	$5.5\times$
6	1,811 s	7,362 s	$4.1\times$

La causa es el tráfico de memoria. Cada operación intermedia en `numpy` materializa un array temporal de dimensión $N \times T$ —unos 38 MB en la configuración medida—, mientras que la versión compilada funde todo el cálculo en una única pasada sin intermedios y lo paraleliza sobre pólizas.

La conclusión es que la compilación JIT aporta más que el cambio de algoritmo, y que ambas mejoras son acumulables: la combinación óptima es `numba` con cuadratura.

6. El caso de fuerza constante

Por completitud se recoge el tratamiento análogo bajo la hipótesis de fuerza constante, que el motor implementa como método alternativo.

Si μ_j no depende de s dentro del período, entonces $1 - q'_j = e^{-\mu_j}$, de donde $\mu_j = -\ln(1 - q'_j)$. La supervivencia conjunta es ${}_sp^{(\tau)} = e^{-\mu s}$ con $\mu = \sum_m \mu_m$, y la integral resulta:

$$(aq)_j = \int_0^1 \mu_j e^{-\mu s} ds = \frac{\mu_j}{\mu} (1 - e^{-\mu}) = \frac{\mu_j}{\mu} \left(1 - \prod_m (1 - q'_m)\right) \quad (20)$$

La constancia del cociente μ_j/μ permite extraerlo de la integral, lo que produce una forma cerrada de coste $\mathcal{O}(K)$ sin desarrollo combinatorio alguno. No existe aquí incentivo para la evaluación numérica: el integrando es exponencial y no polinómico, de modo que la cuadratura sería aproximada, y la forma cerrada ya es óptima.

Esta asimetría explica que el aviso emitido por el motor a partir de seis decrementos sugiera la fuerza constante como alternativa: es la única salida al crecimiento exponencial *mientras el desarrollo de inclusión-exclusión sea la única vía para UDD*.

7. Conclusiones

Sobre la equivalencia. El desarrollo de inclusión-exclusión implementado en el motor no es una definición independiente, sino la solución analítica de la integral (7). Los signos alternados proceden del desarrollo del producto de binomios; los denominadores $1/(|S| + 1)$, de integrar $s^{|S|}$ sobre el intervalo unidad. Ambas formulaciones son matemáticamente idénticas y ninguna aproxima a la otra.

Sobre la exactitud de la cuadratura. El integrando es un polinomio de grado $K - 1$, y la regla de Gauss–Legendre de n nodos es exacta hasta grado $2n - 1$. Con $n = \lceil K/2 \rceil$ la evaluación numérica es exacta salvo redondeo de coma flotante. Se trata, por tanto, de un método alternativo de cálculo y no de una aproximación.

Sobre el coste. La complejidad pasa de $\mathcal{O}(K^2 2^{K-1})$ a $\mathcal{O}(K^3)$, con un ratio asintótico $2^K/K$ confirmado experimentalmente. La mejora medida sobre el motor completo va de $2\times$ con dos decrementos a $5\times$ con seis, y crece sin límite con K .

Sobre la dependencia de numba. Se mantiene necesaria. La ventaja de la compilación supera a la del cambio de algoritmo, y ambas son acumulables.

Propuesta. Intercambiar los papeles de ambos métodos: adoptar la cuadratura como núcleo de cálculo y conservar el desarrollo de inclusión-exclusión como referencia en los tests. La justificación es que se obtiene una implementación más rápida y sin límite práctico en el número de causas, validada contra una formulación algebraica, cerrada y bien comprendida, que a su vez está validada contra hoja de cálculo para el caso de dos decrementos.

Como efecto colateral, el aviso emitido a partir de seis decrementos dejaría de tener fundamento y debería retirarse: con cuadratura, doce causas resultan más baratas de calcular que seis con el método actual.

La única precaución que introduce el cambio es la derivación del número de nodos a partir de K mediante (15). Un número insuficiente degrada silenciosamente la exactitud, por lo que conviene que quede fijado en el código y cubierto por un test explícito.